

Programmation orientée objet : Architecture MVC en PHP + MySQL

Model - View - Controller

Problématique

Quel est le plat préféré des développeurs ?



Du code "spaghetti" : mélange de HTML, PHP et requêtes SQL

Le Code "Spaghetti" (À ÉVITER)

```
<?php
$db = new PDO('mysql:host=localhost;dbname=ecole', 'root', '');
$eleve = $db->query("SELECT * FROM eleves WHERE id = " . $_GET['id']->fetch();
?>

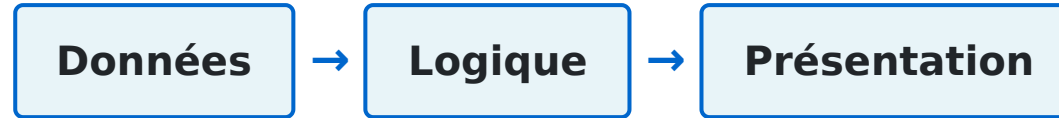
<html>
  <h1><?= $eleve['nom'] ?> <?= $eleve['prenom'] ?></h1>
  <p>Classe: <?= $eleve['classe'] ?></p>
  <p>Option: <?= $eleve['option'] ?></p>
<?php
  if ($_POST['action'] === 'update') {
    $db->query("UPDATE eleves SET nom = " . $_POST['nom'] . " WHERE id = " . $_GET['id']);
  }
?>
</html>
```

Problèmes graves

- Injections SQL : Failles de sécurité critiques
- Code impossible à maintenir : Tout mélangé
- Pas testable : Impossible d'écrire des tests
- Dangereux : Peut causer des accidents graves

Concept de "Separation of Concerns" (SoC)

Séparer les responsabilités du code



- ✓ Chaque partie a une responsabilité unique
- ✓ Facilite la maintenance et l'évolution
- ✓ Permet le travail en équipe
- ✓ Rend le code testable

Le Pattern MVC

 architecture diagram

MVC : Les 3 Couches

MODEL

- Gère les données
- Accès à la base de données
- Logique métier
- Retourne des données

VIEW

- Affiche les données
- Code HTML/CSS
- Reçoit les données du Controller
- Ne fait pas de requêtes SQL

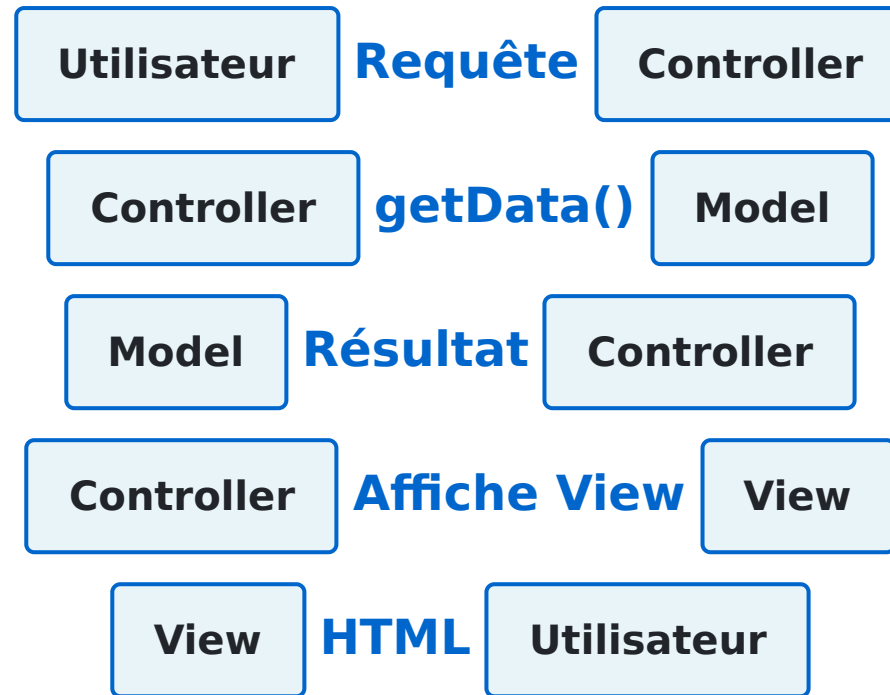
MVC : Les 3 Couches (Suite)

CONTROLLER 🎮

- Reçoit les requêtes utilisateur
- Appelle le Model si nécessaire
- Charge la Vue appropriée
- Fait le lien entre Model et View

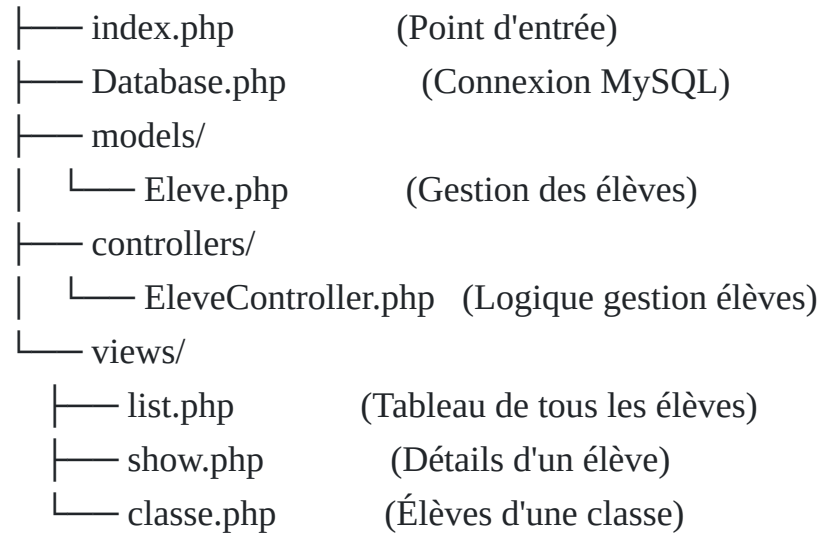
Le Controller est l'orchestrateur !

Flux de Données MVC



Architecture des Fichiers

ecole-app/



Étape : La Connexion (Database.php)

```
<?php
class Database {
    private $pdo;

    public function __construct() {
        $this->pdo = new PDO(
            'mysql:host=localhost;dbname=ecole',
            'root',
            ''
        );
        $this->pdo->setAttribute(
            PDO::ATTR_ERRMODE,
            PDO::ERRMODE_EXCEPTION
        );
    }

    public function query($sql, $params = []) {
        $stmt = $this->pdo->prepare($sql);
        $stmt->execute($params);
        return $stmt->fetchAll(PDO::FETCH_ASSOC);
    }
}
```

Étape : Le Modèle (models/Post.php)

```
<?php
class Eleve {
    private $db;

    public function __construct($database) {
        $this->db = $database;
    }

    // Récupérer tous les élèves
    public function getAll() {
        return $this->db->query(
            "SELECT * FROM eleves ORDER BY nom, prenom"
        );
    }

    // Récupérer un élève par ID
    public function getById($id) {
        $result = $this->db->query(
            "SELECT * FROM eleves WHERE id = ?",
            [$id]
        );
        return $result[0] ?? null;
    }
}
```

Étape : Le Contrôleur (controllers/PostController.php)

```
<?php
class EleveController {
    private $model;

    public function __construct($model) {
        $this->model = $model;
    }

    // Action : afficher la liste
    public function listAction() {
        $eleves = $this->model->getAll();
        require 'views/list.php';
    }

    // Action : afficher les détails d'un élève
    public function showAction($id) {
        $eleve = $this->model->getById($id);
        require 'views/show.php';
    }

    // Action : afficher par classe
    public function classeAction($classe) {
        $eleves = $this->model->getByClasse($classe);
```

Étape 4 : La Vue (views/list.php)

```
<html>
<head><title>Liste des Élèves</title></head>
<body>
<h1>Tous les Élèves</h1>
<table border="1" cellpadding="10">
  <tr>
    <th>Nom</th>
    <th>Prénom</th>
    <th>Date Naissance</th>
    <th>Classe</th>
    <th>Option</th>
    <th>Actions</th>
  </tr>
  <?php foreach ($seleves as $seleve): ?>
    <tr>
      <td><?= htmlspecialchars($seleve['nom']) ?></td>
      <td><?= htmlspecialchars($seleve['prenom']) ?></td>
      <td><?= htmlspecialchars($seleve['date_naissance']) ?></td>
      <td><?= htmlspecialchars($seleve['classe']) ?></td>
      <td><?= htmlspecialchars($seleve['option']) ?></td>
      <td>
        <a href="?action=show&id=<?= $seleve['id'] ?>">Voir</a>
      </td>
    </tr>
  </?>
</body>
</html>
```

Étape : Le Point d'Entrée (index.php)

```
<?php
// 1. Charger les classes
require 'Database.php';
require 'models/Eleve.php';
require 'controllers/EleveController.php';

// 2. Créer la connexion
$db = new Database();

// 3. Créer le modèle avec la BD
$model = new Eleve($db);

// 4. Créer le contrôleur avec le modèle
$controller = new EleveController($model);

// 5. Router simple
$action = $_GET['action'] ?? 'list';
$id = $_GET['id'] ?? null;
$classe = $_GET['classe'] ?? null;

if ($action === 'list') {
    $controller->listAction();
} elseif ($action === 'show' && $id) {
    $controller->showAction($id);
}
```

Flux Complet d'une Requête

URL: index.php?action=show&id=7

1. Router (index.php) → détecte action=show, id=7
2. Controller → appelle model->getById(7)
3. Model → interroge la BDD (SELECT * FROM eleves WHERE id = 7)
4. BDD → retourne les données de l'élève #7
5. Model → retourne (nom, prénom, date naissance, classe, option)
6. Controller → charge views/show.php avec l'élève
7. Vue → affiche les détails complets de l'élève
8. Navigateur → reçoit le HTML avec toutes les informations

Avantages de MVC

- ✓ **Maintenance** : Changer la vue ne touche pas au modèle
- ✓ **Testabilité** : On peut tester le Model indépendamment
- ✓ **Réutilisabilité** : Un Model peut servir plusieurs Vues
- ✓ **Évolutivité** : Facile d'ajouter de nouvelles fonctionnalités
- ✓ **Travail en équipe** : Designer sur les Vues, Dev sur Model/Controller
- ✓ **Pas de code "spaghetti"** : Chaque partie a sa place

MVC vs Code Spaghetti

Aspect	Spaghetti	MVC
Localisation	SQL partout	SQL au même endroit (Model)
Évolution	Risqué	Sûr et isolé
Test	Impossible	Facile pour chaque partie
Équipe	Conflits	Travail parallèle
Performance	Peut être rapide	Très rapide si bien conçu
Lisibilité	Horrible	Excellent

Pièges à Éviter

-  Mettre de la logique dans la Vue
-  Faire des requêtes SQL dans le Controller
-  Retourner un objet compliqué du Model
-  Charger plusieurs Modèles dans un Controller
-  Oublier de nettoyer les entrées utilisateur
-  Garder chaque part simple et responsable

Prochaines Étapes

1. **Autoloader PSR-4** : Charger les classes automatiquement
2. **Router avancé** : Gérer les URLs propres (/posts/5)
3. **Services** : Ajouter une couche métier
4. **Dépôt** : Centraliser les requêtes BD
5. **Framework** : Laravel, Symfony, Slim

Questions ?

Vous avez compris le pattern MVC ?

Prochaine étape : construire votre première app MVC ! 🚀